

C++ Vectors: Complete Reference Sheet

A structured guide covering theory, functions, complexity, and copy-paste ready C++ code.

1 Introduction to Vectors

A **vector** in C++ is a dynamic array provided by the Standard Template Library (STL). Unlike standard static C-style arrays, vectors grow and shrink dynamically as elements are added or removed.

- 1. **Dynamic Sizing:** Automatically resizes its capacity when elements are inserted or deleted.
- 2. **Contiguous Memory:** Elements are stored in sequence, enabling fast direct indexing and cache friendliness.
- 3. **Random Access:** Provides constant time $O(1)$ access to elements via indexing.

2 Header and Initialization

To use vectors, include the `<vector>` header file.

Vector Initialization Examples

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    // 1. Uninitialized empty vector (size = 0)
    vector<int> vec1;

    // 2. Vector with initial size 5 (elements initialized to 0)
    vector<int> vec2(5);

    // 3. Vector with size 5, all elements set to 10
    vector<int> vec3(5, 10);

    // 4. Initializer list syntax (C++11+)
    vector<int> vec4 = {10, 20, 30, 40, 50};

    // 5. Initialize by copying another vector
    vector<int> vec5(vec4);

    return 0;
}
```

3 Element Access Functions

Functions used to read or modify vector elements at specific positions:

Function	Description	Time Complexity
<code>vec[i]</code>	Returns reference to element at index <code>i</code> (no bounds check).	$O(1)$
<code>vec.at(i)</code>	Returns reference at index <code>i</code> (with bounds check; throws <code>out_of_range</code>).	$O(1)$

Function	Description	Time Complexity
<code>vec.front()</code>	Returns reference to the first element.	$O(1)$
<code>vec.back()</code>	Returns reference to the last element.	$O(1)$
<code>vec.data()</code>	Returns pointer to the underlying array used internally by vector.	$O(1)$

Element Access Code

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> numbers = {10, 20, 30, 40, 50};

    std::cout << "First element (front): " << numbers.front() << "
"; // Output: 10
    std::cout << "Last element (back): " << numbers.back() << "
"; // Output: 50
    std::cout << "Element at index 2 (at): " << numbers.at(2) << "
"; // Output: 30
    std::cout << "Element at index 2 ([]): " << numbers[2] << "
"; // Output: 30

    return 0;
}
```

4 Modifier Functions

Functions to insert, remove, or modify element contents:

Function	Description	Time Complexity
<code>push_back(val)</code>	Appends element to the end.	$O(1)$ amortized
<code>pop_back()</code>	Removes last element.	$O(1)$
<code>insert(pos, val)</code>	Inserts element at specified iterator position.	$O(N)$
<code>erase(pos)</code>	Removes element at specified iterator position.	$O(N)$
<code>clear()</code>	Removes all elements from vector.	$O(N)$
<code>emplace_back(args)</code>	Constructs element directly at end in-place.	$O(1)$ amortized

Modifiers Code

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> vec;

    vec.push_back(10);
    vec.push_back(20);
    vec.push_back(30);

    vec.pop_back(); // Removes 30

    vec.insert(vec.begin() + 1, 15); // [10, 15, 20]
    vec.erase(vec.begin());          // [15, 20]
    vec.clear();                      // Vector empty

    return 0;
}
```

5 Capacity & Size Functions

Understanding internal sizing vs allocated capacity:

1. **Size:** Actual count of elements currently stored.
2. **Capacity:** Total element memory slots allocated before reallocation is needed.

Function	Description
<code>vec.size()</code>	Returns number of current elements.
<code>vec.capacity()</code>	Returns allocated memory capacity.
<code>vec.empty()</code>	Returns true if size is 0.
<code>vec.resize(n)</code>	Resizes vector to hold <code>n</code> elements.
<code>vec.reserve(n)</code>	Allocates memory ahead of time to avoid reallocations.

6 Iteration Methods

Looping Techniques

```

#include <iostream>
#include <vector>

int main() {
    std::vector<int> vec = {10, 20, 30, 40};

    // 1. Index-based loop
    for (size_t i = 0; i < vec.size(); ++i) {
        std::cout << vec[i] << " ";
    }
    std::cout << "
";

    // 2. Range-based for loop (C++11)
    for (int val : vec) {
        std::cout << val << " ";
    }
    std::cout << "
";

    // 3. Iterator-based loop
    for (auto it = vec.begin(); it != vec.end(); ++it) {
        std::cout << *it << " ";
    }
    std::cout << "
";

    return 0;
}

```

7 2D Vector (Vector of Vectors)

Dynamic 2D grid structure using vectors.

2D Vector Example

```

#include <iostream>
#include <vector>

int main() {
    int rows = 3, cols = 4;
    std::vector<std::vector<int>> matrix(rows, std::vector<int>(cols, 0));

    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            std::cout << matrix[i][j] << " ";
        }
        std::cout << "
";
    }

    return 0;
}

```

8 Time Complexity Summary

1. Random Access: $O(1)$
2. Insertion/Deletion at end: $O(1)$ amortized
3. Insertion/Deletion at middle or start: $O(N)$
4. Linear Search: $O(N)$
5. Binary Search (Sorted Vector): $O(\log N)$